SSL, or the Web's Pillar of Authentication and Confidentiality:

Past, Present and Future

Luis Grangeia, luis.grangeia@gmail.com

Tiago Mendo, tiagomendo@gmail.com

What is SSL?

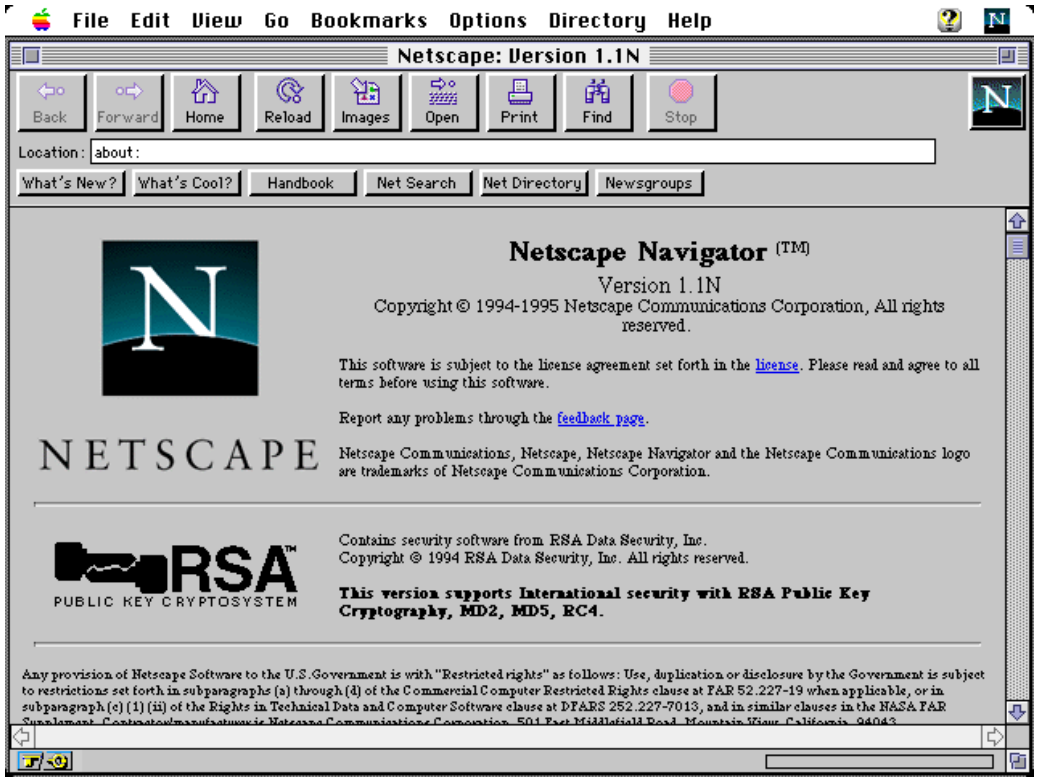
Once upon a time…

History of SSL

- Developed to protect communications between web browser and servers

- 1st version by Netscape in 1994

- 18 years old!



History of SSL

- Followed by SSL 2.0 (1995) and SSL 3.0 (1996)

- IETF standardized the protocol
  - SSL (**S**ecure **S**ockets **L**ayer) to TLS (**T**ransport **L**ayer **S**ecurity)
  - TLS 1.0 (1999), 1.1 (2006) and 1.2 (2008)

- SSL 3.0 and TLS 1.0 are equivalent in security

History of SSL

"Everybody knows SSL. TLS is more technically accurate but sounds like a cable TV network or a disease"

-Brad Hill, Black Hat Briefings USA 2007

History of SSL: the PKI

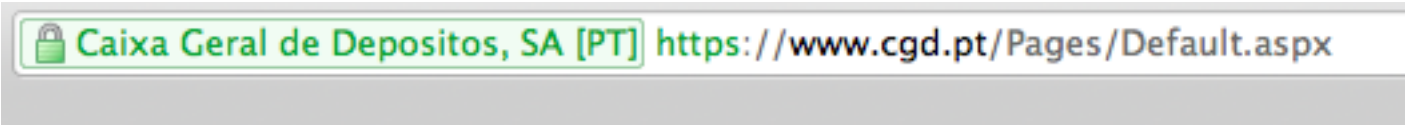
- PKI – **P**ublic **K**ey **I**nfrastructure
  - Hierarchy of trust anchors: X.509 certificates issued by CAs
- X.509 dates from 1988
  - 23 year old model!
- SSL requires X.509 certificates with a PKI

What does SSL gives us?

- **Confidentiality** – encrypts data in transit
- **Integrity** – uses MACs to detect tampering
- **Authentication** – public key cryptography to authenticate peers (X.509 certificates)
- All **dependent** on the PKI trust model

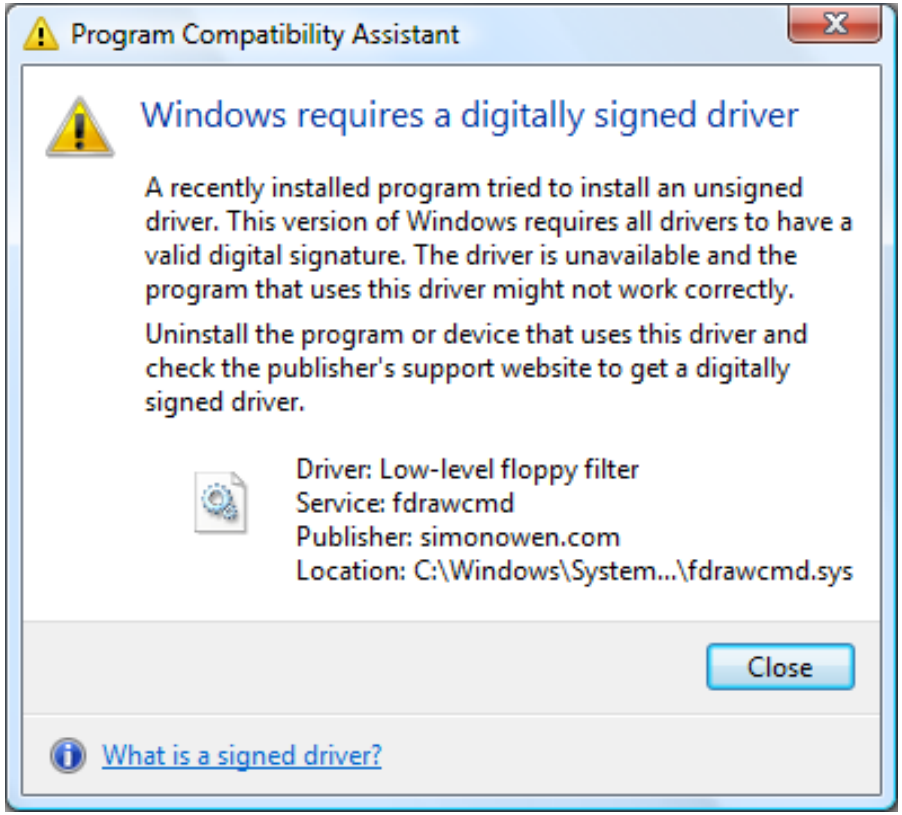
Examples of usage

- **HTTPS**
  - Provides secure web communication
    - Protects credentials and content from eavesdroppers
    - Ensures data is not tampered



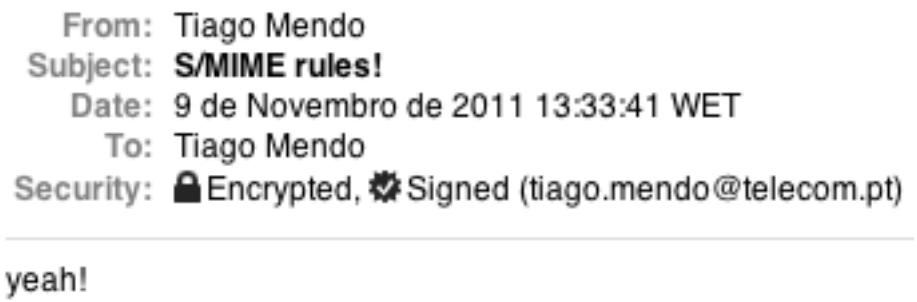
Examples of usage

- **Code signing**
  - Securely distribute software and updates



Examples of usage

- **Email certificates**
  - Secure email communications with S/MIME
    - Digitally signed
    - Encrypted



Examples of usage

- **Communication tunnels**
  - VPN
    - OpenVPN uses SSL extensively
  - stunnel
    - Multi-platform tool that provides SSL to peers that do not support it natively

Examples of usage

- **Mutual authentication**
  - Client authenticates itself to the server with a X.509 certificate
  - Common situations:
    - HTTPS
    - IM
    - S/MIME
    - VPNs

Problems

Problems

- SSL has problems
  - Many of them are technical problems AKA bugs
  - And most of them can be easily solved

Old SSL versions

- Example: SSL 2.0
  - Dumb-down attack
    - cipher downgrade to 40 bits
  - Padding
    - Padding-length field not authenticated
    - Increase value -> trim messages from the end
  - Easily solved -> disable SSL 2.0 at client and server

End user devices

- **Implementation bugs**:
  - OS X v10.6.8 June 2011 update changelog:

"*An error handling issue existed in the Certificate Trust Policy. If an Extended Validation (EV) certificate has no OCSP URL, and CRL checking is enabled, the CRL will not be checked and a revoked certificate may be accepted as valid. This issue is mitigated as most EV certificates specify an OCSP URL.*"

End user devices

- **Implementation bugs**:
  - iOS 4.3.5 July 2011 update changelog:

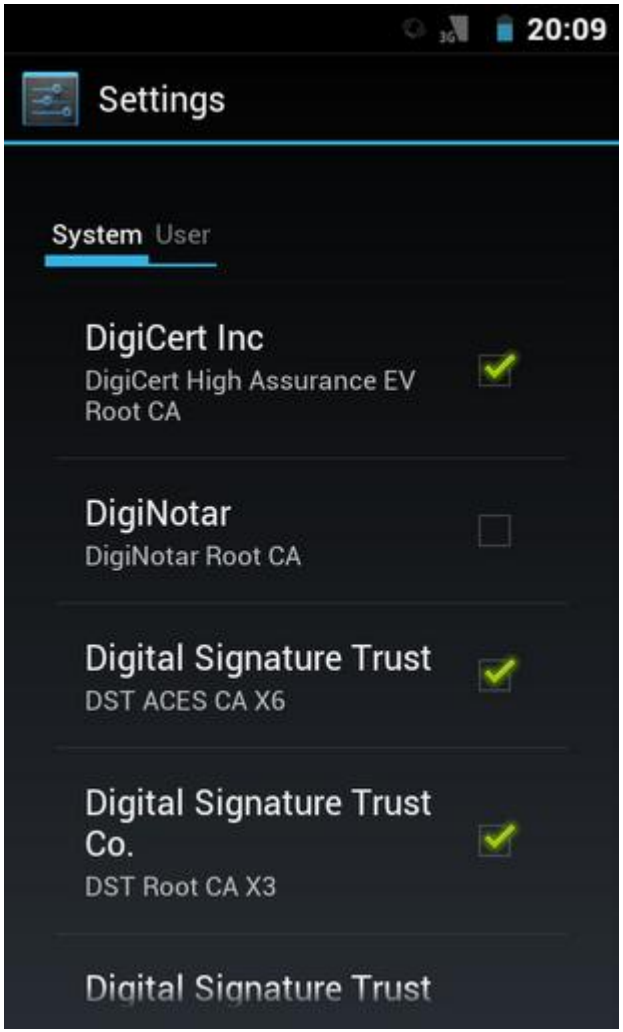
    "*A certificate chain validation issue existed in the handling of X.509 certificates. An attacker with a privileged network position may capture or modify data in sessions protected by SSL/TLS.*"

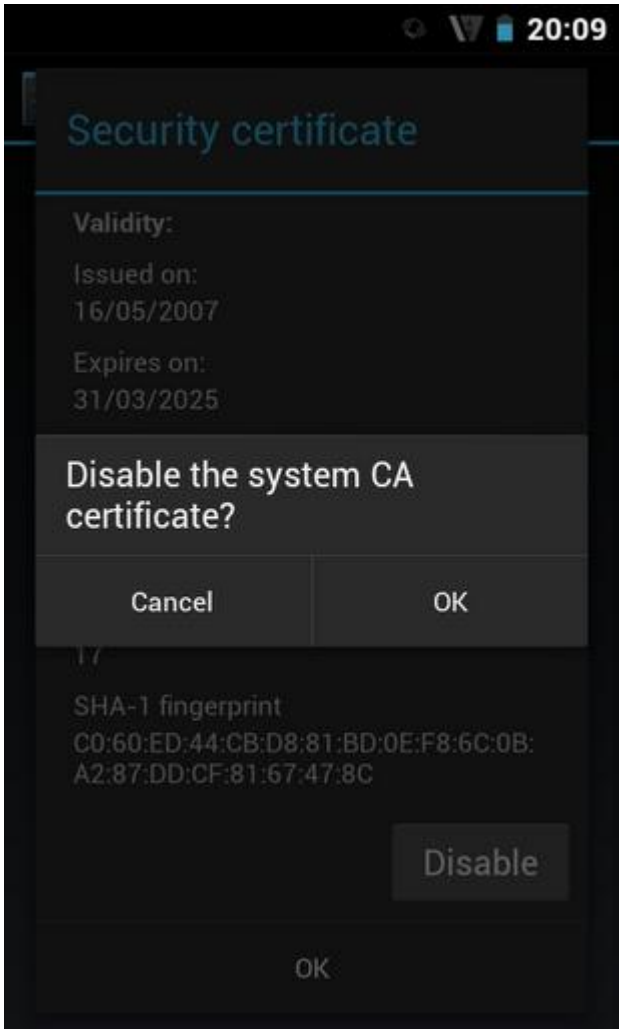
  - Translates to "I can issue any certificate I want."

End user devices

- Even without bugs, you are missing features
  - Grab your pocket computer (yes, that thing that does phone calls)
  - Try to remove a root certificate.
  - Even if you can…which one do I remove?

(in Android ICS)





What about those bad certificates?

Revocation

- CRL - **C**ertificate **R**evocation **L**ist
  - “here, have a list of bad certificates”

- OCSP - **O**nline **C**ertificate **S**tatus **P**rotocol
  - “is this certificate still good?”

Revocation… is hard

- Servers can be unreachable
- Browsers ignore CRL and OCSP errors

  – Does this option really matter?

- Browsers cache responses
  – For months.
  – So…

Revocation – the 3rd method

- Update every browser in the planet.
  - Release new version
  - Test, approve, pack
  - Download
  - Install
  - Close the browser
  - Reboot the computer

- OCSP Stapling?

Go to your homebanking

Did you typed HTTPS
Or HTTP?

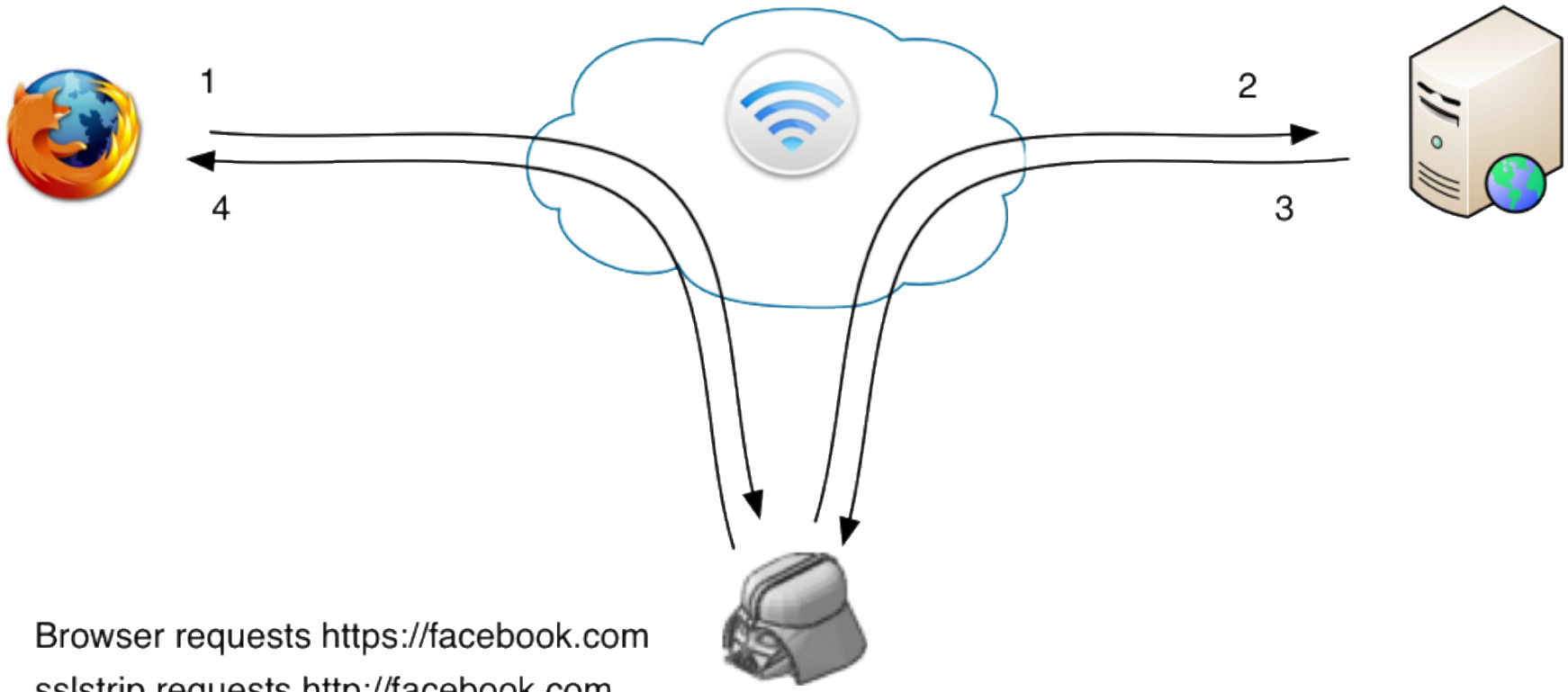
First hit

- First hit on HTTPS sites is usually…HTTP

“I 0wn you “

First hit problems - sslstrip

- s/https/http/g



1 Browser requests https://facebook.com
2 sslstrip requests http://facebook.com
3 Server answers with http://facebook.com
4 sslstrip servers fake https://facebook.com

First hit solutions- HSTS

- **H**TTP **S**trict **T**ransport **S**ecurity
  - Server defined policy that browsers must honor

GET login.sapo.pt 200 OK log

Headers Response Cache HTML

Response Headers view source

Date	Fri, 11 Nov 2011 10:52:40 GMT
Server	Apache
Vary	Cookie,Accept-Encoding
Cache-Control	no-cache
Pragma	no-cache
Expires	Wed, 31 Dec 1969 23:59:59 GMT
Content-Language	pt-PT
P3P	policyref="http://login.sapo.pt/P3I
Strict-Transport-Security	max-age=15768000;includeSubdomains

- "Browser, convert all requests to my domain to HTTPS"

Certificate issuance

- The "Microsoft Corporation" certificates

- The "is this yours?" certificates

  "I have not been able to find the current owner of this root. Both RSA and VeriSign have stated in email that they do not own this root."

  – one of the maintainers of Mozilla CA list

- Insecure Security? Why?

It this a SSL or PKI problem?

- Seems a PKI problem
  - These are design and structural problems
  - Hard to solve
    - Like replacing the pillars of a bridge without collapsing

The Trust Model of SSL today

Or “what is a PKI”

SSL: The Trust Model

1. Root Certificate Authorities (CAs) **rule.**
2. CAs **sell** certificates to entrust companies and users.
3. CAs **delegate trust** to other CAs **for a price** (so they can do the same, creating a "chain of trust").
4. There is a **validation process** when they sell you a certificate.

SSL: The Trust Model
end user certificate validation (simplified)

1. **Server**: "this is my certificate, here is a chain of trust leading up to a root CA"

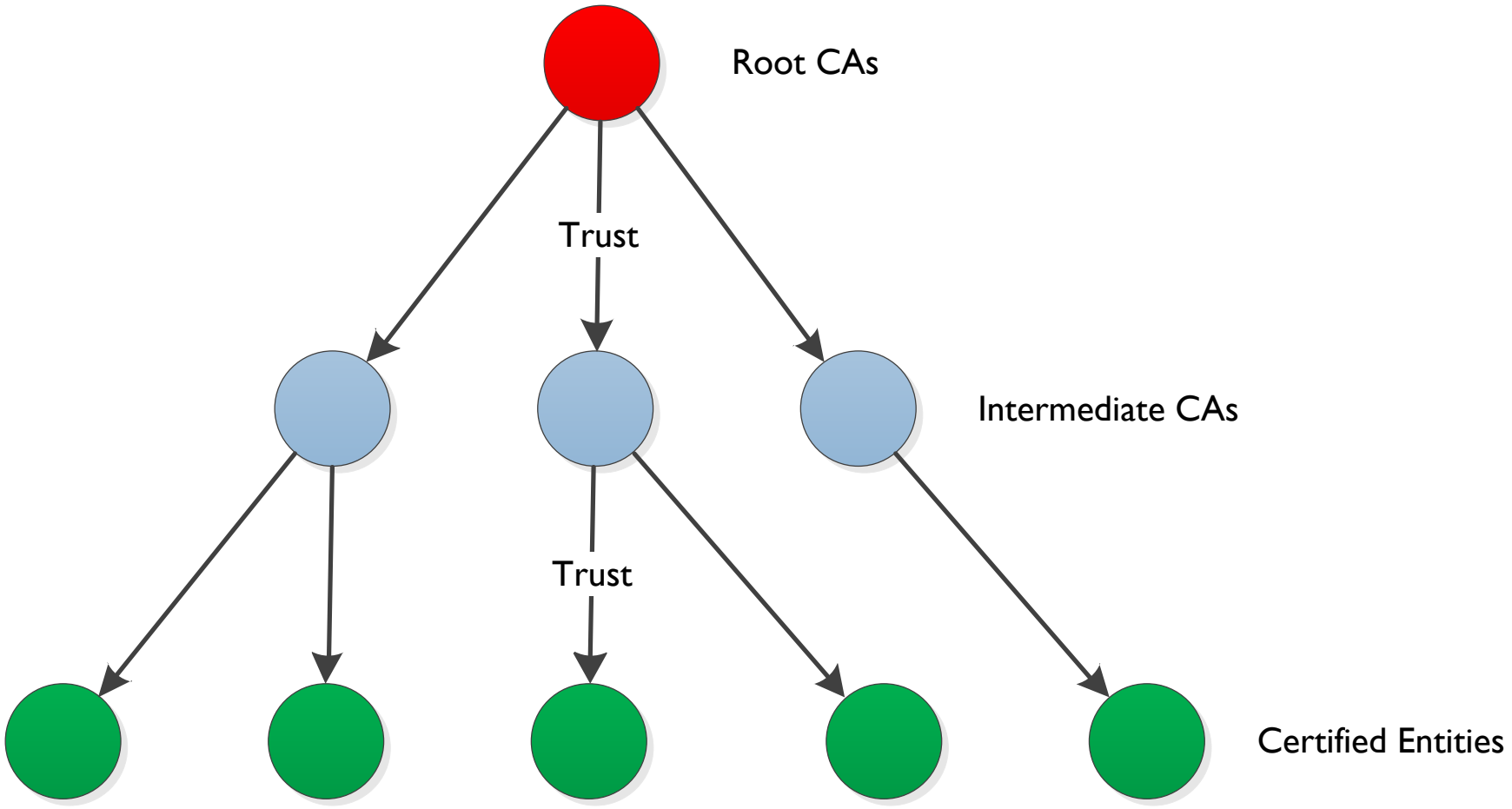
2. **Client**: "let's see if any of these certificates have been revoked/expired…" (using OCSP, or checking a local CRL)

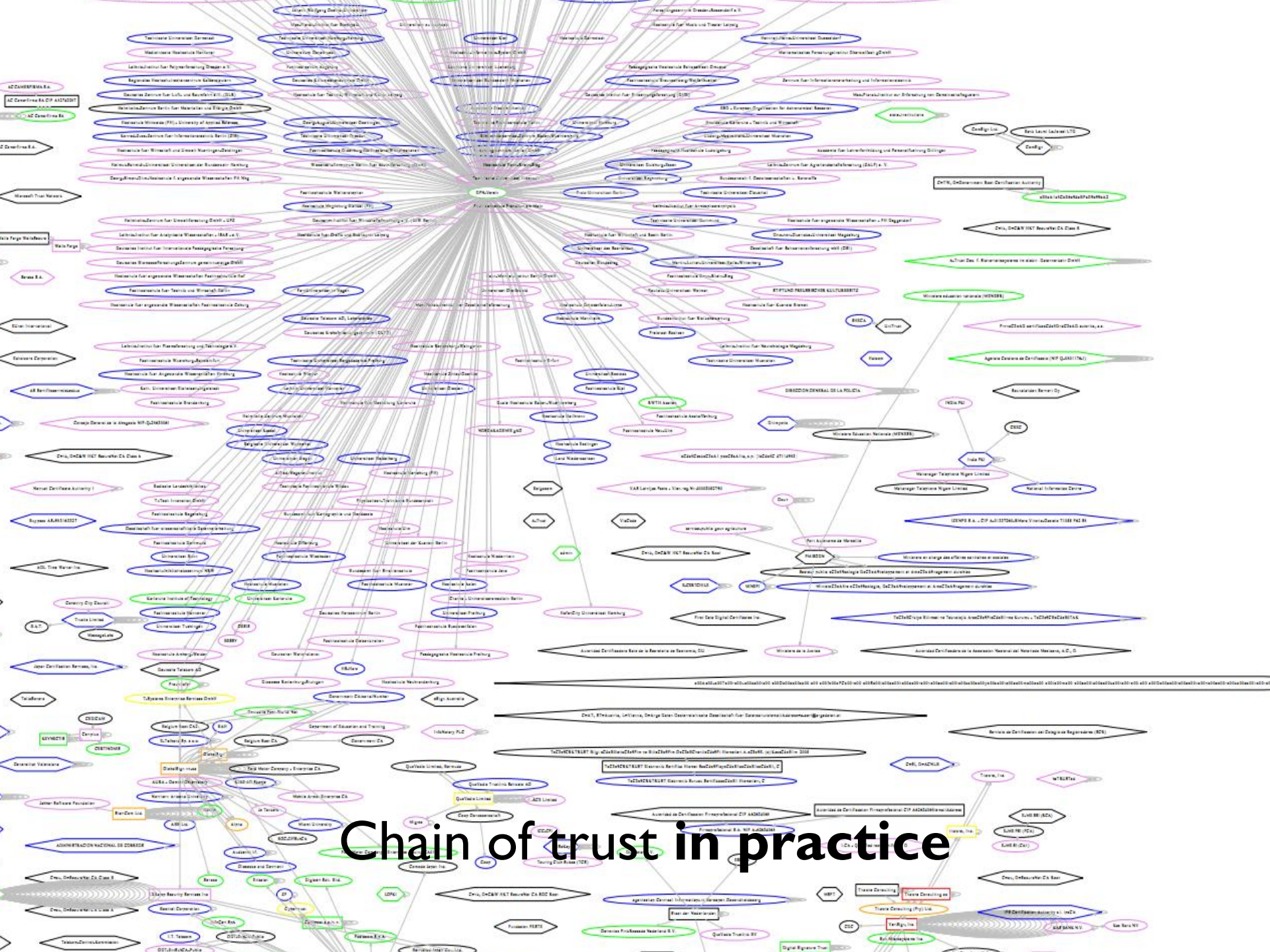
3. **OK**: "The story adds up, all the certificates in the chain are valid and linked to a trusted CA"

   Or…

4. **NOT OK**: "I don't trust that CA, or some of these certificates have been revoked/expired, or the name of the certificate doesn't match the hostname, etc."

Chain of trust (in principle…)



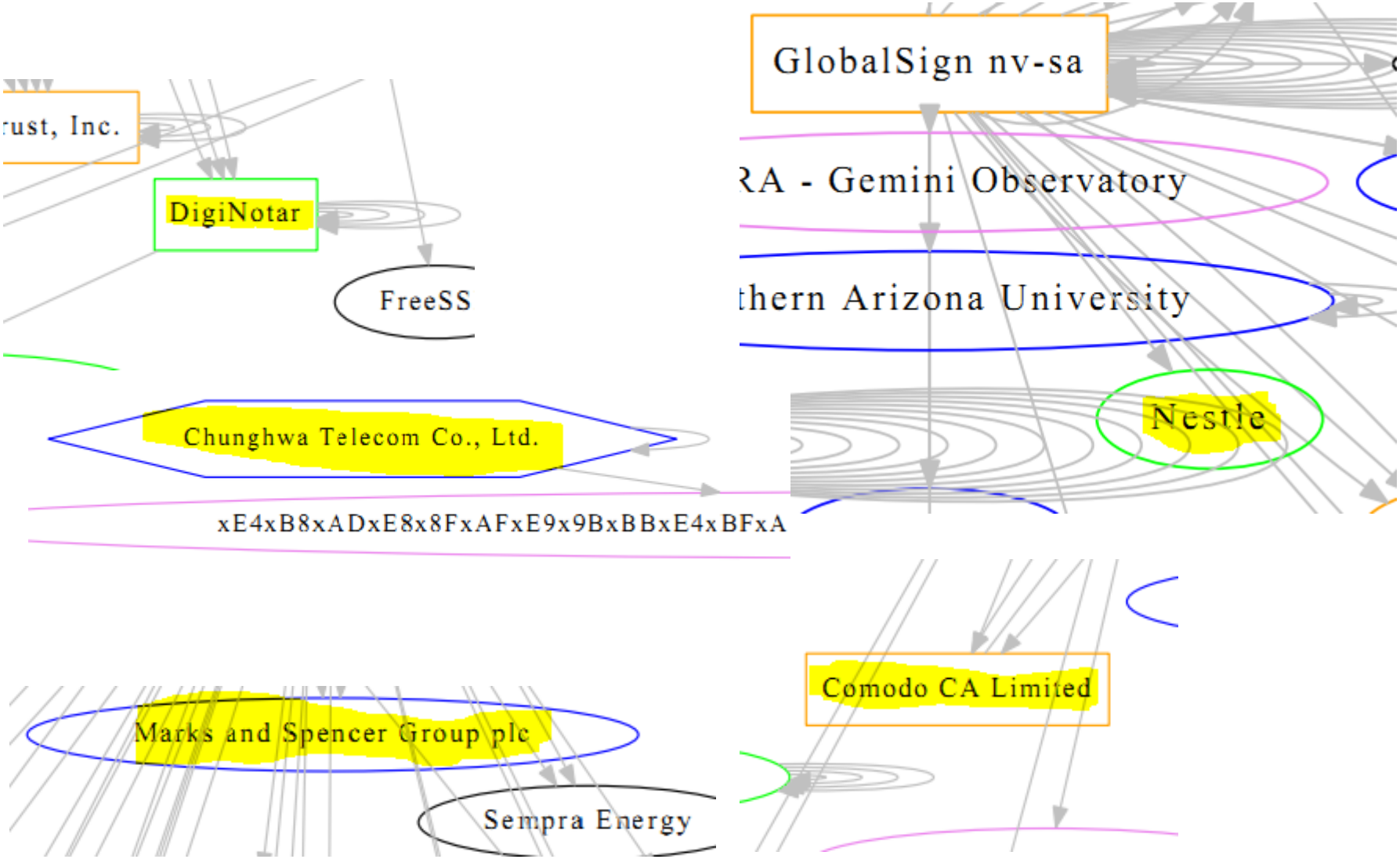


Chain of trust **in practice**

Source: **EFF Observatory**

- **Data from 2010**
- **Around 650 organizations in the chain of trust of Microsoft and Mozilla**
- Map Key:
  - Hexagon: root CA trusted by Microsoft only
  - Diamond: root CA trusted by Mozilla only
  - Box : root CA trusted by both
  - Ellipse: subordinate CA

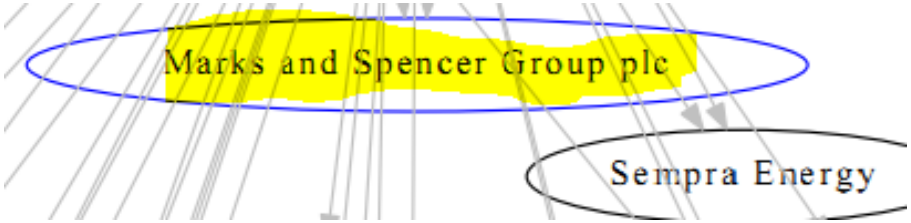
  - Black : signed 0 leaves
  - Violet: signed 1-10 leaves
  - Blue : signed 11-100 leaves
  - Green : signed 101-1000 leaves
  - Yellow: signed 1001-10000 leaves
  - Orange: signed 10001-100000 leaves
  - Red : signed 100001-1000000 leaves


GlobalSign nv-sa
rust, Inc.
DigiNotar
RA - Gemini Observatory
FreeSS
thern Arizona University
Nestle
Chunghwa Telecom Co., Ltd.
xE4xB8xADxE8x8FxAFxE9x9BxBBxE4xBFxA
Comodo CA Limited
Marks and Spencer Group plc
Sempra Energy

The Trust Model of SSL today sucks.

Why?

The security of the Internet depends on the security of "Marks & Spencer"…



Current Trust Model Flaws

- **650+ "single points of failure":**
  - CAs are hacked all the time, even root CAs:
    - Comodo, StartSSL, **Diginotar**
  - Malicious insiders
  - Malicious CAs selling certificates to governments
  - CAs are **profit-driven**, creating a **"race to the bottom"** in regards to **reducing costs verifying authenticity** of certificate requestors

Next up…

- DigiNotar hack aftermath;
- Atacker created “forged” certificates for Google, Facebook, Twitter;
- Might have sold them to Iranian government (details not clear);
- Next video shows real time use of these stolen certificates (data collected using OCSP queries made to DigiNotar server).

Operation Black Tulip
2011-07-30 00:00:00
FOX-IT
EXPERTS IN IT SECURITY

Remember, this is **IRAN**, people.

What about:

China

Russia

USA?

There are 26 Computer Forensics from at least 60 suppliers on Alibaba.com

Related Searches: Computer Forensics, forensic, computer cleaner, laptop computer, Wholesale Computer Forensics

See larger image: HTTPS/SSL Network Packet Forensics Device - HTTPS/SSL Real-Time Forensics and Lawful Interception System

Add to My Favorites

HTTPS/SSL Network Packet Forensics Device - HTTPS/SSL Real-Time Forensics and Lawful Interception System

Other products by Products Status: Stock

FOB Price:	US $ 1,000 - 20,000 Get Latest Price
Port:	Taiwan
Minimum Order Quantity:	1 Unit/Units
Payment Terms:	T/T

Mr. Kan Chang
Offline

Contact Supplier
Send a Message to this Supplier

Supplier Details

Decision Group Inc
[Taiwan]
Business Type:
Manufacturer
Contact Details

Online Showroom: 4 Products
Other Similar Products from this Supplier
View this Supplier's Website

Report Suspicious Activity

Current Trust Model Flaws (2)

- The Model is **Monolithic**
  - Since the list of root CAs is centrally maintained, every user is affected the same way;
- It is also, **Not Agile**:
  - Hacks are very hard to recover from, especially to root CAs: Diginotar was compromised in June 2011, it is still accepted as trusted by virtually the entire world.
- The Model is **Money and Power driven,** not user driven:
  - Whoever has the more money and influence, gets to choose what can be trusted (big companies, governments.)

So, this is bad…

…so bad in fact, that Google is already starting to work around it:

"Starting with Chrome 13, ***we'll have HTTPS pins for most Google properties*** *[sites]. This means that certificate chains for, say, https://www.google.com, must include a whitelisted public key.* ***It's a fatal error otherwise****."*

"The whitelisted public keys for Google currently include Verisign, Google Internet Authority, Equifax and GeoTrust. Thus ***Chrome will not accept certificates for Google properties from other CAs****."*

- Adam Langley, Google

"Every man for himself."

The Trust Model of SSL today sucks.

Now what?

“Trust us.”

“Trust us.”

Organizations vs Individuals

In general:

- **Organizations are evil:** They only care about reputation if it brings an advantage (profit, votes, customers);
- **Individuals are good:** We are biologically programmed to be honorable and to build a good reputation (everyone wants to be right).

Would you rather trust a corporation (even a government) or your close group of friends?

Would you like to have a **choice**?

Right now, **you don't**.

Or do you?

Solutions

“Trust is not bought. It is earned.”

"You can fool some of the people all of the time, and all of the people some of the time, ***but you cannot fool all of the people all of the time****."*

- Abraham Lincoln

Solutions

- **Decentralize** the Trust Model:
  - Trust decisions should ultimately be the result of **individual user choice**, not imposed. The model should provide information so that every user decides in the most informed way.

- **Democratize** the Trust Model:
  - Give it the ability to take into account the **trust decisions of individuals**;

Solutions: they're on the way

Alternatives already exist:

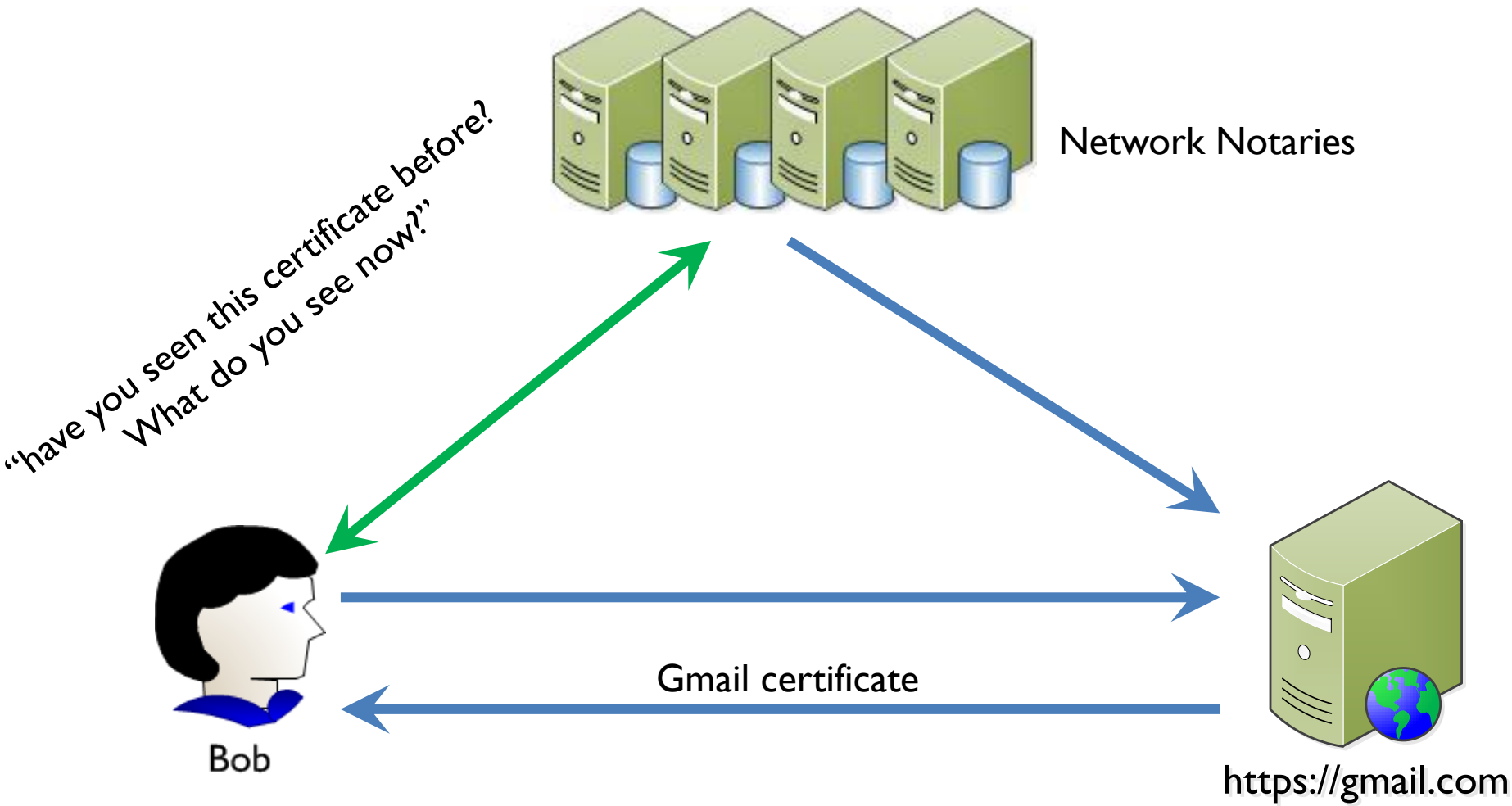
- **Perspectives Project**: Based on academic research from Adrian Perrig (Carnegie Mellon)
  - http://perspectives-project.org/

- **Convergence**: Similar implementation by Moxie Marlinspike
  - http://convergence.io/

Similar approaches

- **Replace** CAs with **notaries**;
- Notaries **do not certify**, they just **validate**;
- A user can **consult** with a subset, or all the **notaries**;
- The user can **decide based** on the **results** from notaries (accept only a unanimous decision, a majority decision, etc.)

Network Notary Architecture



Advantages

- Can work alongside current PKI (although Convergence chooses not to);
- Notaries can provide a timeline of changes of certificates;
- Provide network perspective against based Man-in-the-Middle attacks (remember Iran and Diginotar).

Problems

- Notaries must be authenticated first;
- No incentive for notaries to “behave”;
- Does not address certificate revocation;
- Privacy problems for users (notaries can look at your web browsing habits).

We **thought** of a different **solution**.

Premises

- **Do not impose** a specific model – We're going to provide the tools to help the user decide who/when to trust, the final decision can be his. This means we should be able to play well with the current PKI model or the newer notary model.
- **Design is user centric** – User initiates trust relations and decides when to strengthen or terminate them.
- **Locality** – As in the real world, the "closer" someone is to you, the easier for you to trust them. Use this to our advantage.
- **Versatility** – You can use our model to trust individuals or sites/corporations

Locality breeds trust

Compare This:

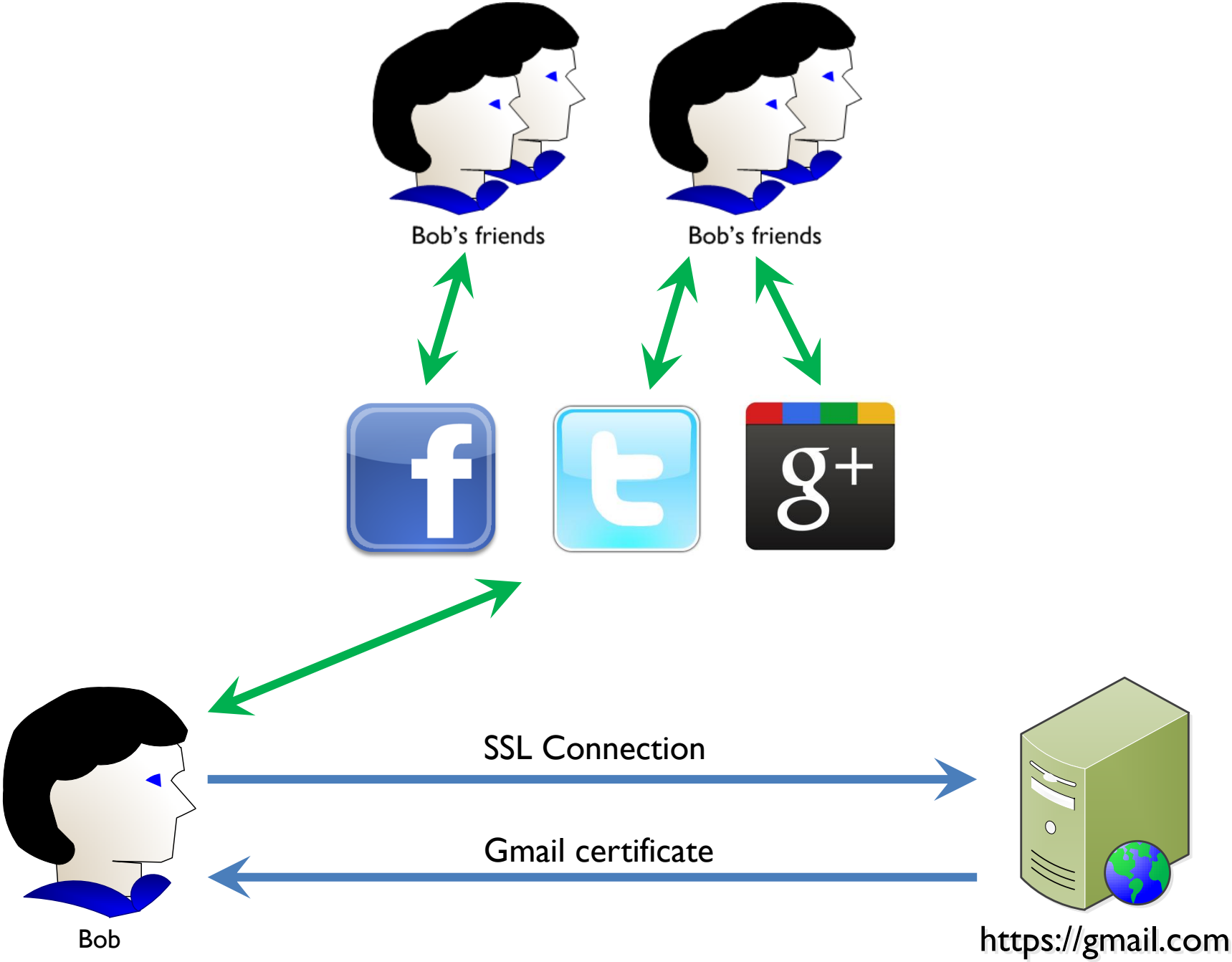
Process of accepting a friend request on Facebook

To This:

Process of issuing a SSL certificate with Verisign

Design Decisions

- **Explore social presence and connections** (aka "friendsourcing"): We'll attempt to capitalize on the increasing number of social links "declared" on the Internet;
- **Explicit user feedback**: Only share trust relations between individuals/sites if the user wants to do it (prevent privacy issues);
- **Use Social Networks as a transport medium**: Treat them as an untrusted medium but a way to easily broadcast messages to your friend circles.


Bob's friends
Bob's friends
SSL Connection
Gmail certificate
Bob
https://gmail.com

- **“Is this certificate OK? Let’s see…”**
  - Check my friend’s walls for references to this certificate;
  - Check PKI’s chain of trust and other certificate parameters;

- **Possible Answers:**
  - *“The chain of trust is valid. Also, 4 of your friends have visited this site recently and vouched for it”*
  - *“while there is a valid chain of trust,* ***none of your friends seen this certificate before****. Also,* ***2 of your friends have seen a different certificate for this site****”*
  - *“There is a valid chain of trust, but* ***4 of your friends consider this certificate forged*** *and do not recommend you to continue.”*

Compare readability to this…

Use Case 1

1. **Bob** wants to **access gmail.com** for the first time on this computer.
2. Bob's **browser** contacts gmail.com, **gets the certificate**.
3. Bob's **browser** looks at Bob's various **social networks** to see what his friends have **asserted** about that certificate (signed XML messages on friends Walls, for instance).
4. **Using that information**, as well as PKI **chain of trust data**, Bob **chooses** to **accept and save** that certificate for subsequent accesses.

Use Case 2

1. After using Gmail.com for a couple of times, Bob is pretty confident that it is the real deal, so he chooses to **vouch for that certificate**, and **display** that information **on** his **Facebook Wall**.
2. The message is in **XML** (for machine parsing), **signed** using his personal certificate, and displayed only for his select group of friends (or to the public, if he chooses so).
3. The message contains: serial and hash of Gmail certificate, date of access, level of trust assigned (**negative trust is possible too**).

Bonus Feature

- You can also reinforce trust in your friends by signing their certificates and vouching for them;
  - **Does this seem familiar?**

Remember PGP's "Web of Trust"?

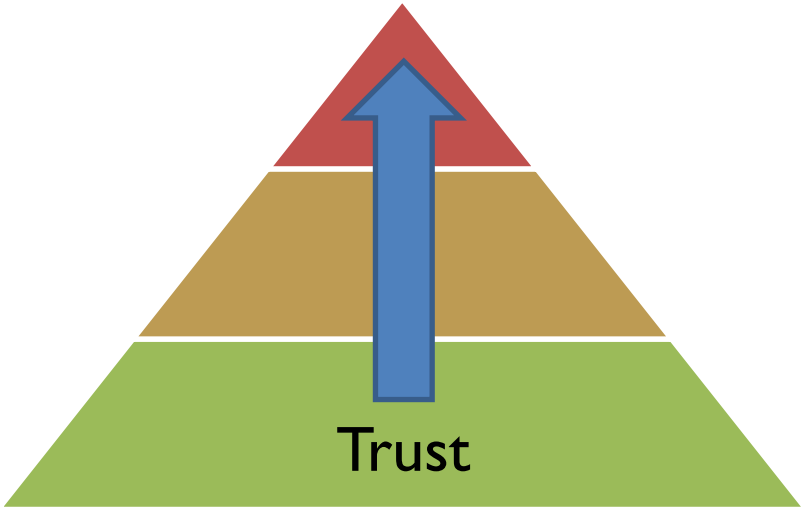
(aka the oldest social network of the Internet.)

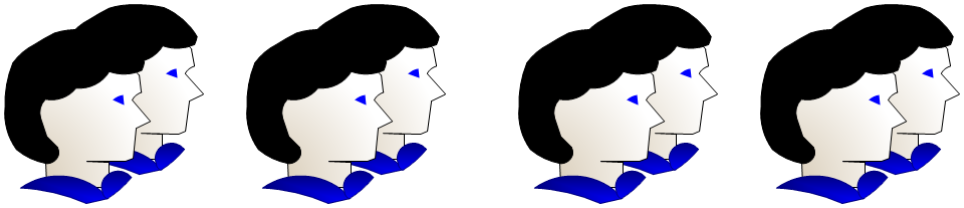
Final Thoughts

- SSL has a bunch of protocol flaws, that are being addressed:
  - BEAST attack (with cipher suite changes);
  - SSLstrip (with HSTS);
  - Renegotiation bug (upgrade protocol version);
- But all of these flaws **pale in comparison** with how much the PKI trust model **sucks**.
- To change the trust model, we need to **change the paradigm**.

We want to **change the paradigm**

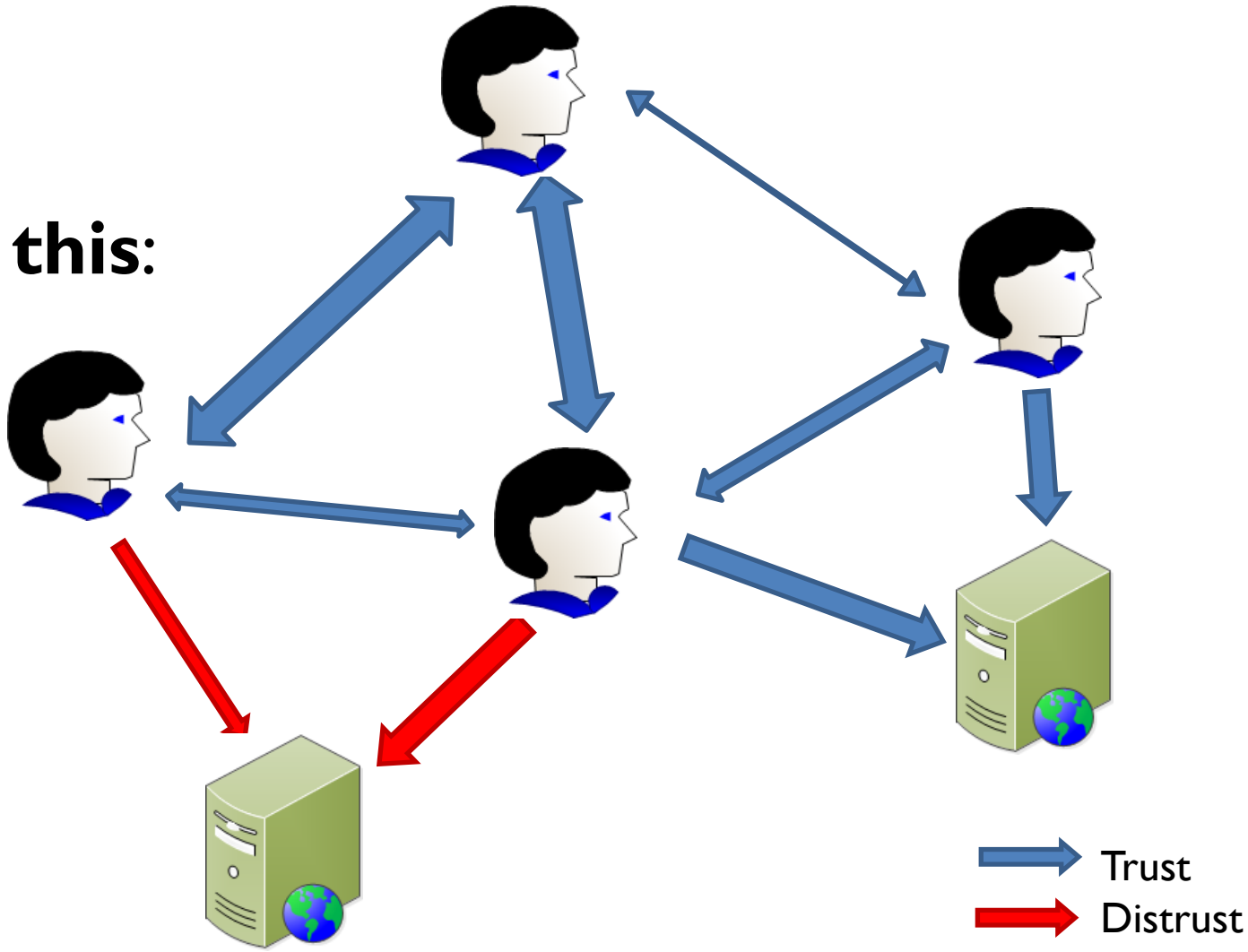
from **this**:





We want to **change the paradigm**

to **this**:



Conclusion

- SSL Trust Model is **Broken.**
- Make it **less** centered **on "the 1%"** and **more** centered **on "the 99%"**.
- Help us turn this idea into practice!

Web of Trust + Social Networks =

New Trust Model for the Internet

Thank you!
Q&A

Luis Grangeia, luis.grangeia@gmail.com
Tiago Mendo, tiagomendo@gmail.com